



Movie App

A Vue 3 Client-Side Single-Page Application

Semester Project Deliverable 1

Client-Side Architecture (Vue 3 + Vite)

Student Name:	Umair Azhar / Abdul Aziz
Roll No :	23010101-044 / 23010101-023
Course:	Web Engineering
Instructor:	Sir. Museb
Date:	02 July, 2026

1. Introduction

This Vue App is an movie website made with Vue 3. A user can look through a list of movies, filter them by genre, search for a movie by its name, open a page to see the full details of any movie and save movies to their own watchlist. instead of getting data from a server or the internet it uses a small list of movies already stored inside the project (src/data/movies.js). Because of this the app does not need an API key or internet to run.

1.1 Technologies used

Tool	Why it is used
Vue 3	The main framework written in the modern <script setup> style.
Vite	The build tool and fast local development server.
Vue Router	Moves the user between pages without reloading the browser.
Pinia	Stores the shared watchlist data in one central place.
HTML / CSS	Basic page structure and simple styling.

1.2 What the app can do

- **Home:** look through all the movies, filter them by genre and go to the search page.
- **Search:** type a movie name and see the matching movies with the matched words highlighted.
- **Movie details:** see the rating, runtime, director, genres, cast and summary of one movie.
- **Watchlist:** save the movies you like also the list stays saved even after the page is refreshed.

2. Single-Page Application (SPA) Routing

A Single-Page Application loads one HTML page just once and then changes the content as the user moves around. The page never fully reloads, so the app feels fast and smooth. Vue Router is what makes this possible. All the routes (pages) are listed in one file, src/router/index.js. Each route connects a URL to a Vue component (a page):

URL	Page component	What it shows
/	HomePage.vue	All movies + the genre filter
/search	SearchPage.vue	Search results by movie title
/movie/:id	MovieDetailPage.vue	Full details of one movie
/watchlist	WatchlistPage.vue	The user's saved movies
anything else	redirects to '/'	Catch-all for unknown URLs

2.1 Important routing details

- **Route parameter:** The `:id` part in `/movie/:id` changes for each movie (for example `/movie/1` or `/movie/5`). The detail page reads this value using `useRoute().params.id` to know which movie to show.
- **Catch-all route:** The last route `/notFound(.*)` sends the user back to the home page if they type a URL that doesn't exist, instead of showing a broken page.
- **Clean URLs:** The router uses `createWebHistory()`, which gives normal, clean URLs without a `#` symbol.

2.2 Layout wrappers: `<RouterView>` and `<RouterLink>`

Two special Vue Router tags are used to build the page layout:

- **`<RouterView>`** : placed inside `App.vue`. It works like a window that always shows whichever page matches the current URL. When the URL changes, `<RouterView>` swaps in the correct page while the Navbar above it stays in place.
- **`<RouterLink>`** : used instead of a normal `<a>` tag. It takes the user to another route without reloading the whole page. It is used in the Navbar (Home / Search / Watchlist) and inside every movie card (to open that movie's detail page).

Navigation from code: sometimes we need to change pages from inside JavaScript instead of from a link. On the Home page when the user types something and clicks "Search", the code runs `router.push('/search?q=' + searchText.value)` which takes the user to the search page and sends the typed word along in the URL.

3. Modular Component Architecture

Instead of writing the whole app in one big file the app is split into many small reusable pieces called components. Each component is a `.vue` file that has its own template (HTML) and its own logic (JavaScript) and it can be reused wherever it is needed. This keeps the code clean and easy to understand.

The project has a clear folder structure inside the `src/` directory:

```
src/
├── data/
│   └── movies.js      # built-in list of movies (our "data")
├── components/
│   ├── Navbar.vue     # the top menu
│   └── MovieCard.vue  # one movie card <- props + emits example
├── pages/              # one component per page (route)
│   ├── HomePage.vue
│   ├── SearchPage.vue
│   ├── MovieDetailPage.vue
│   └── WatchlistPage.vue
├── stores/
│   └── watchlist.js    # Pinia store for saved movies (+ localStorage)
├── router/
│   └── index.js        # the list of pages / routes
├── App.vue            # layout: Navbar + the current page
└── main.js            # starts the app (Vue + Pinia + Router)
```

3.1 Role of each part

File / folder	Role
main.js	Starting point: creates the app and connects Pinia and the Router.
App.vue	The overall layout: shows the Navbar on top and the current page below.
router/index.js	The list of all pages (routes).
components/	Small reusable pieces used by many pages (Navbar, MovieCard).
pages/	One component for each route (Home, Search, Detail, Watchlist).
stores/	The Pinia store that holds the shared watchlist data.
data/movies.js	The built-in list of movies (used instead of a database).

3.2 Two levels of components

- **Components (components/ folder)** : small pieces that are reused. Navbar.vue is the top menu shown on every page. MovieCard.vue shows one movie as a small card and is reused on both the Home page and the Watchlist page.
- **Pages (pages/ folder)** : each one is a full page tied to a route. A page puts the smaller components and the data together to build a full screen.

Since MovieCard.vue is written once but used in two places (Home and Watchlist) it is a good example of a reusable component. Each page simply loops through its own list of movies and shows one MovieCard for each one.

4. Inter-Component Communication (Props & Custom Events)

Components often need to talk to each other. In Vue data can move in two directions:

- **Props** : sends data DOWN from a parent component to a child component.
- **Emits (custom events)** : sends messages UP from a child component to its parent.

The MovieCard.vue component is a clear example of both directions working together.

4.1 Props: data going down

A card needs to know which movie to show and if that movie is already saved. The parent sends this information in as props. Inside the card the props are set up with type safety so Vue checks that the right type of data is passed in:

src/components/MovieCard.vue

```
const props = defineProps({
  movie: { type: Object, required: true },
  isSaved: { type: Boolean, default: false },
})
```

- **Movie**: must be an Object and is required (the card can't work without it).
- **isSaved**: must be a Boolean and defaults to false when it is not given.

4.2 Emits: events going up

The card has an “Add to watchlist” / “Remove from watchlist” button. But the card itself does not decide what happens that is the parent’s job. So when the button is clicked the card simply sends out an event called toggle along with the movie:

src/components/MovieCard.vue

```
const emit = defineEmits(['toggle'])

function save() {
  emit('toggle', props.movie) // notify the parent (event goes UP)
}
```

4.3 The parent listens

The parent page (Home or Watchlist) sends the props down and listens for the toggle event. When the event happens the parent tells the watchlist store to add or remove the movie:

src/pages/HomePage.vue (and WatchlistPage.vue)

```
<MovieCard
  :movie="movie"
  :is-saved="watchlist.isSaved(movie.id)"
  @toggle="watchlist.toggle($event)"
/>
```

Here, :movie and :is-saved are props (data going down) while @toggle listens for the child’s event (event going up). \$event is the movie sent back by the card. This completes the two-way communication the parent gives data to the child and the child tells the parent when the user does something.

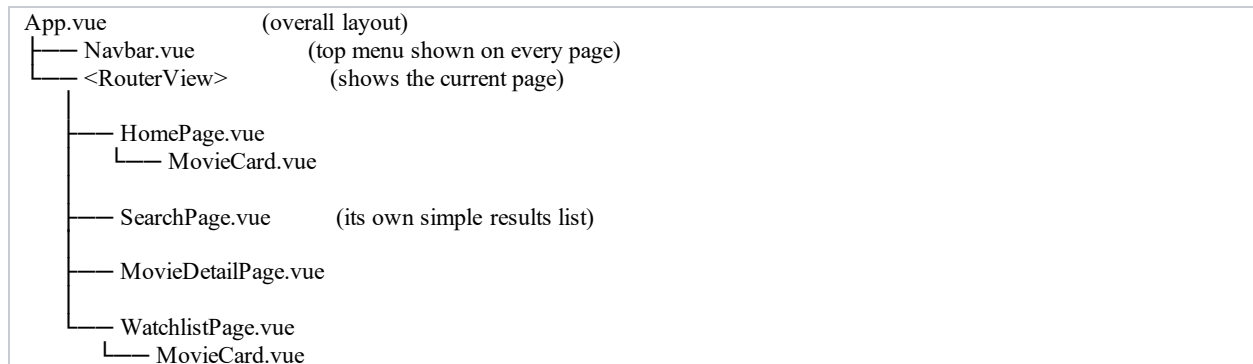
5. Additional Vue Features Used

This deliverable mainly focuses on routing, components, and props/emits, but the project also uses a few other important Vue features:

- **Shared state with Pinia (stores/watchlist.js)** : one central store keeps track of the saved movies. When a movie is added or removed every part of the app that uses it (the Navbar count and the Watchlist page) updates right away. Every change is also saved to the browser’s localStorage so the watchlist stays saved even after a page refresh.
- **Computed properties** : the Home page builds its filtered movie list and the Search page builds its results list using computed() so they update automatically whenever their inputs change.
- **Watchers** : the Search page uses watch() to keep the search box matched with the URL and the Movie Detail page uses watchEffect() to update the browser tab’s title with the movie’s name.
- **Template refs** : the Home page uses a ref to automatically focus the search box as soon as the page loads.
- **Template directives** : the templates use v-for (loops), v-if / v-else (conditions), v-model (two-way form binding), v-bind (:), v-html and {{ }} for showing text.

6. Architecture Tree

The diagram below shows the Vue component structure. App.vue is at the top it always shows the Navbar and uses <RouterView> to swap in the current page. Both the Home and Watchlist pages reuse the MovieCard component (shown as ×N, meaning one card for each movie).



7. Code Snippets

7.1 The router file

The router file lists every route and connects each URL to its page component. It also has a catch-all route that sends unknown URLs back to the home page.

src/router/index.js

```
import { createRouter, createWebHistory } from 'vue-router'
import HomePage from '../pages/HomePage.vue'
import SearchPage from '../pages/SearchPage.vue'
import MovieDetailPage from '../pages/MovieDetailPage.vue'
import WatchlistPage from '../pages/WatchlistPage.vue'

const routes = [
  { path: '/', component: HomePage },
  { path: '/search', component: SearchPage },
  { path: '/movie/:id', component: MovieDetailPage },
  { path: '/watchlist', component: WatchlistPage },
  { path: '/*', redirect: '/' },
]

const router = createRouter({
  history: createWebHistory(),
  routes,
})

export default router
```

7.2 A component using props and emits

MovieCard gets its data through props and sends the button click back to its parent through an emit. Both parts are shown below.

src/components/MovieCard.vue

```
<script setup>
const props = defineProps({
  movie: { type: Object, required: true },
  isSaved: { type: Boolean, default: false },
})

const emit = defineEmits(['toggle'])

function save() {
  emit('toggle', props.movie)
}
</script>

<template>
<div>
  <h3>
    <RouterLink :to="/movie/" + movie.id">{{ movie.title }}</RouterLink>
  </h3>
  <p>Year: {{ movie.year }}</p>
  <p>Rating: {{ movie.rating }}/10</p>
  <button @click="save">
    <span v-if="isSaved">Remove from watchlist</span>
    <span v-else>Add to watchlist</span>
  </button>
  <hr />
</div>
</template>
```

7.3 The parent passing props and listening for the event

src/pages/HomePage.vue

```
<MovieCard
  :movie="movie"
  :is-saved="watchlist.isSaved(movie.id)"
  @toggle="watchlist.toggle($event)"
/>
```

8. Screenshots (Running on localhost)

The screenshots below show the app running locally with `npm run dev` (usually at `http://localhost:5173`). Replace each box with your own screenshot.

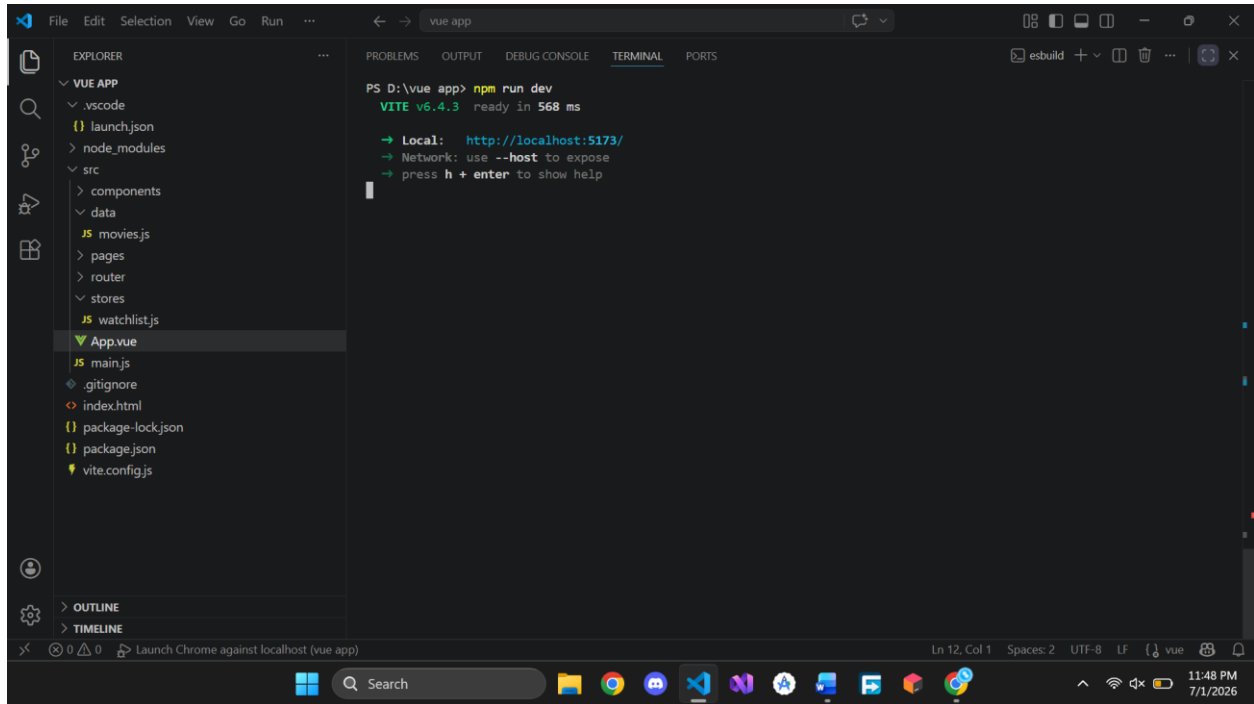


Figure 1 Terminal: the dev server running and printing the local address.

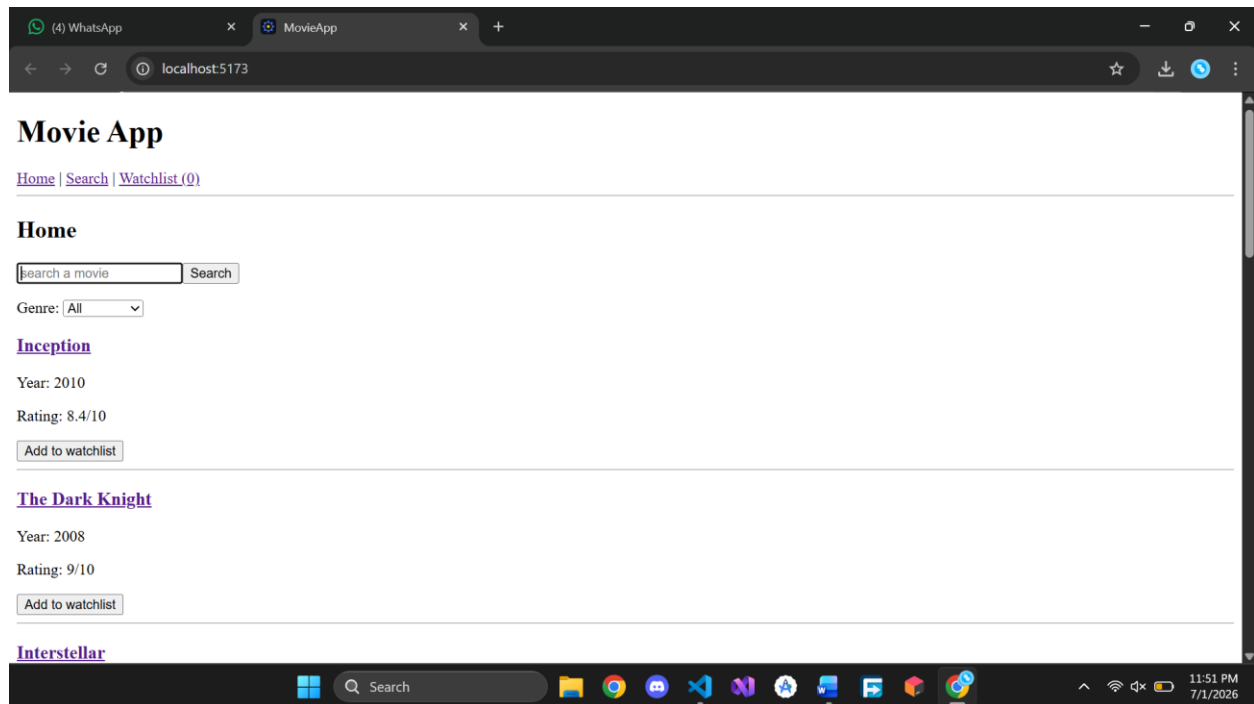


Figure 2 Home page: the movie list with the genre filter and search bar.

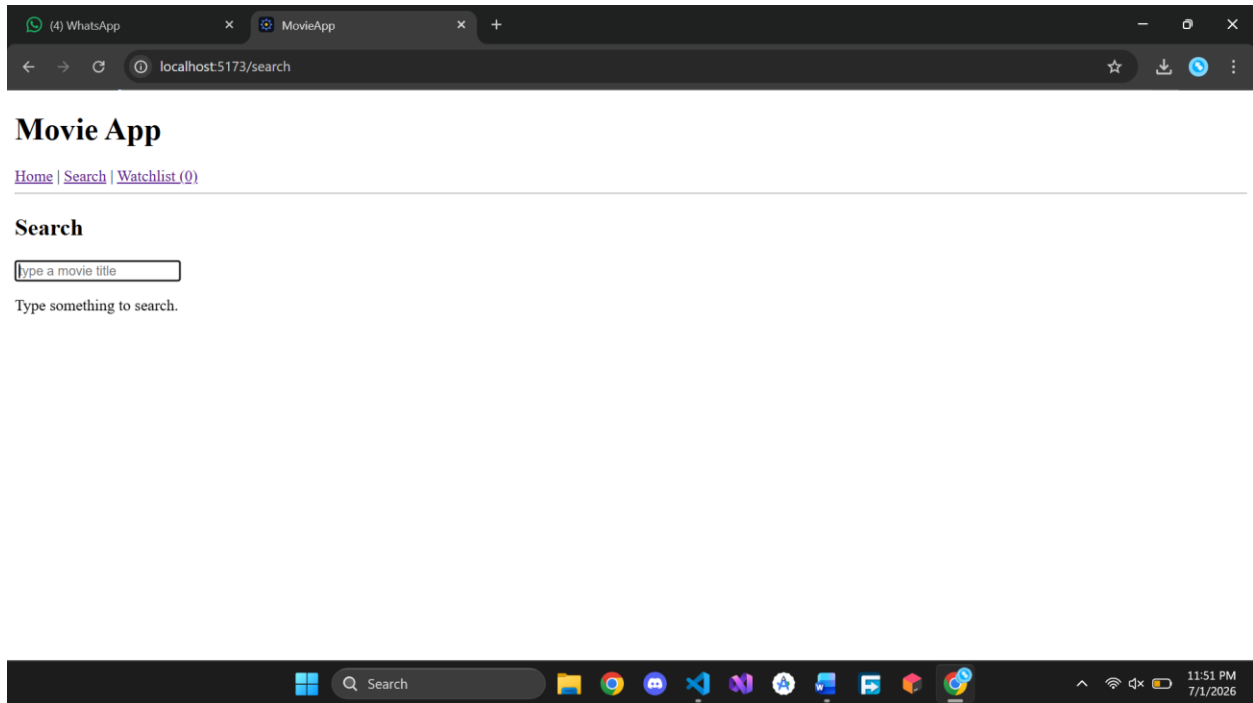


Figure 3 Search page: results for a title, with the matched text highlighted.

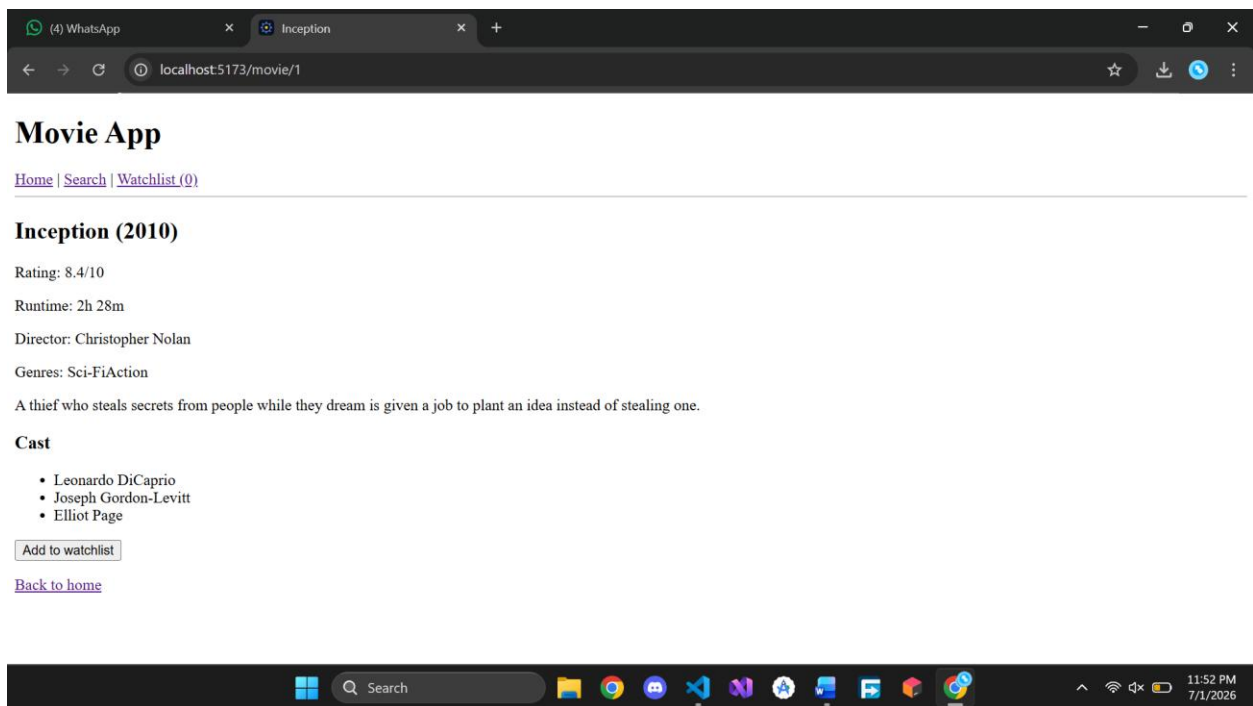


Figure 4 Movie detail page: rating, cast, and overview of one movie.

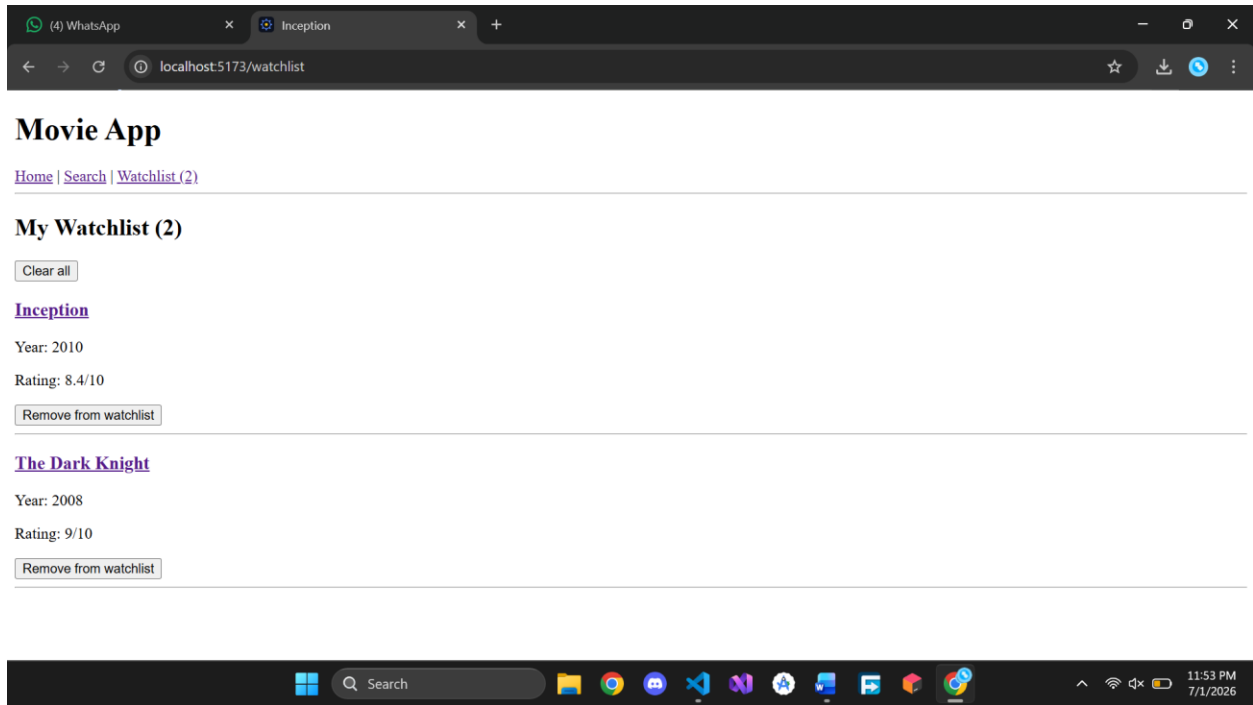


Figure 5 Watchlist page: the saved movies (remembered after refresh).